# 《算法分析与设计》

# 实验报告

学　　号 2024112639

姓　　名 毛露涵

年　　级 2024 级

专　　业 软件工程

院　　系 计算机与人工智能学院

| 实验预习（40%） | 上机实验（40%） | 文档排版（20%） | 总分 | 批阅人签字 |
|---|---|---|---|---|
| | | | | |

2025-2026 学年第 2 学期

# 实验 3.3 任务分配问题

## 1. 实验目的

（1）理解穷举法典型算法的求解过程。

（2）学习穷举法的时间复杂度分析方法，并通过实验验证算法的执行效率。

（3）学会如何利用穷举法求解具体问题，了解穷举法的局限性。

## 2. 实验任务

（1）利用穷举法手工完成实验 3.3 测试用例的求解。

（2）用 C++语言实现该算法并进行测试。

（3）撰写实验报告，实验报告内容包括实验目的、实验任务、实验环境、实验步骤、实验结果和实验总结等部分。

## 3. 实验环境

### 3.1 硬件环境

（1） 计算机：Legion R9000P ADR10H

（2） CPU: AMD Ryzen 9 8945HX with Radeon Graphics

（3） RAM：32GB

### 3.2 软件环境

（1） 操作系统：Windows11 庭中文版

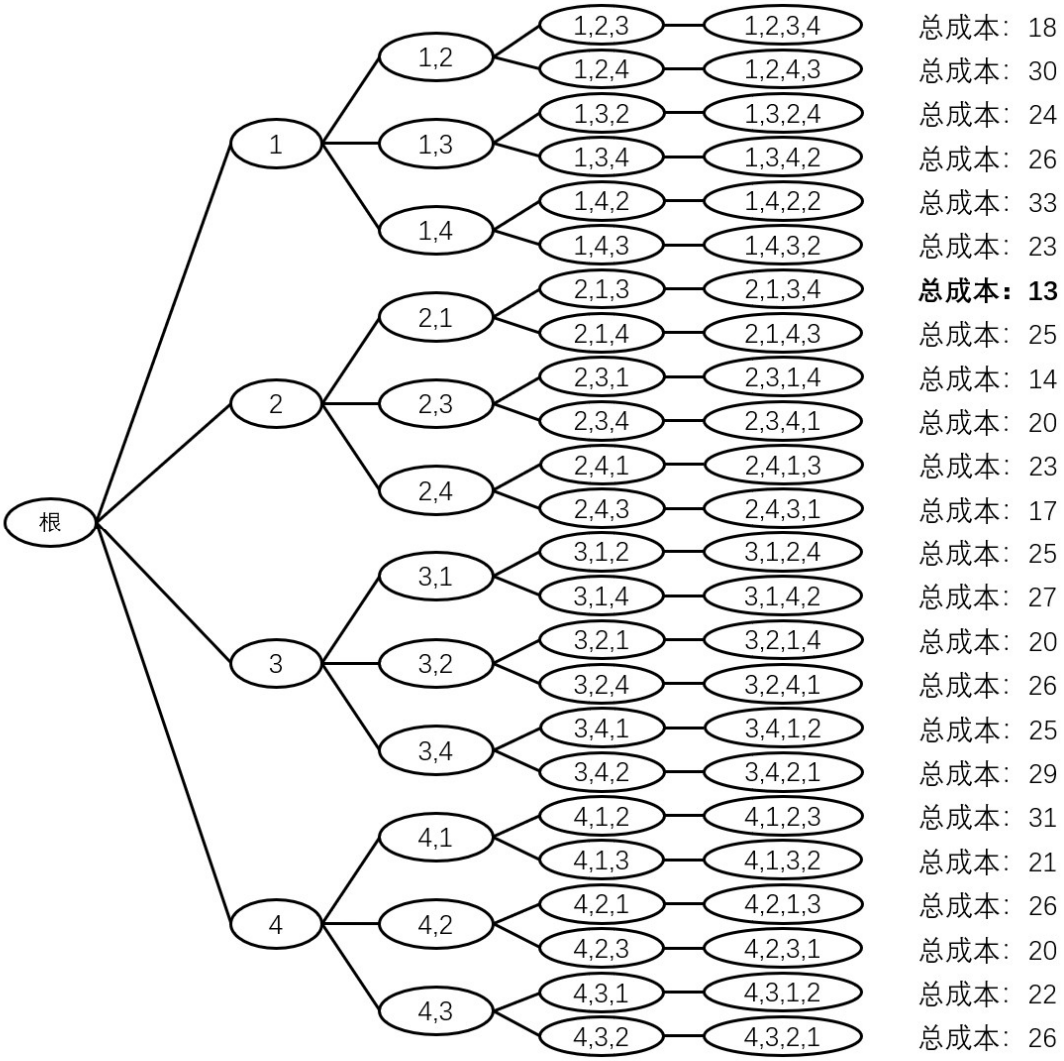
（2） 开发工具：Visual Studio 2026

## 4. 实验步骤及结果

### 4.1 实验预习

#### 4.1.1 根据样例输入采用穷举法求出一种总成本最小的分配方案

1. 排列树




2. 约束条件：x[i] != x[j](i != j)

目标函数：min sum = c[1][x[1]] + c[2][x[2]] + c[3][x[3]] + c[4][x[4]]

3. 最小总成本：13

一种最优分配方案：

第 1 个人：任务 2

第 2 个人：任务 1

第 3 个人：任务 3

第 4 个人：任务 4

### 4.1.2 用 C/C++语言实现的源代码

```
#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
```

```
int n;
int c[20][20];
bool used[20];
int ans = INF;

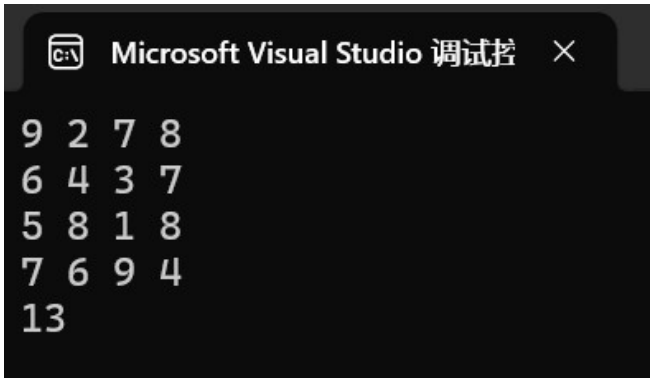
void dfs(int person, int sum)
{
    if (person == n)
    {
        ans = min(ans, sum);
        return;
    }
    for (int task = 0; task < n; task++)
    {
        if (!used[task])
        {
            used[task] = true;
            dfs(person + 1, sum + c[person][task]);
            used[task] = false;
        }
    }
}

int main()
{
    string line;
    getline(cin, line);
    stringstream ss(line);
    int x;
    vector<int> first;
    while (ss >> x)
    {
        first.push_back(x);
    }
```

```
        n = first.size();
        for (int j = 0; j < n; j++)
        {
            c[0][j] = first[j];
        }
        for (int i = 1; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                cin >> c[i][j];
            }
        }
        dfs(0, 0);
        cout << ans << endl;
        return 0;
    }
```

### 4.2 上机实验

#### 4.2.1 上机调试，验证你的求解过程是否正确，写出验证过程




程序输出的最小总成本为 13，与手工计算得到的最小总成本一致，可以验证程序的求解过程是正确的

#### 4.2.2 设计新的测试用例，对程序进行进一步验证

测试输入：

9 2 8 7 6 5

6 7 3 9 8 4

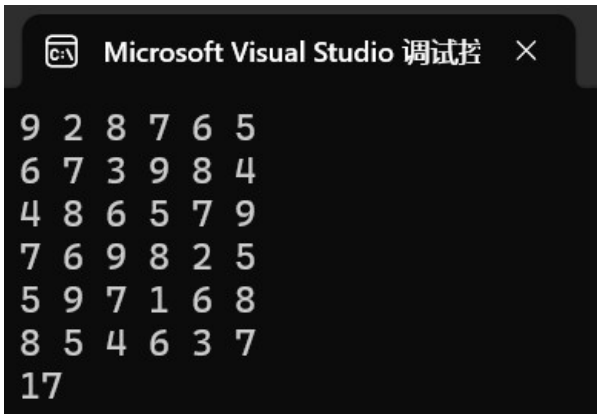
4 8 6 5 7 9

7 6 9 8 2 5

5 9 7 1 6 8

8 5 4 6 3 7

程序执行结果：




对该测试用例进行分析，一种最优分配方案为：
第 1 个人：任务 2，成本为 2
第 2 个人：任务 6，成本为 4
第 3 个人：任务 1，成本为 4
第 4 个人：任务 5，成本为 2
第 5 个人：任务 4，成本为 1
第 6 个人：任务 3，成本为 4
该方案的总成本为：2+4+4+2+1+4=17
手工计算结果与程序运行结果一致，可以验证算法正确。

## 5. 完成实验 3.2 的实验预习 1

当有 n 个元素时，第一个位置有 n 种选择；每选择一种后，还需要继续对剩下的 n-1 个元素进行全排列。因此，它的时间复杂性递推公式为：T(n)=nT(n-1)+O(n)
其中，O(n)表示每一层中循环和交换元素等操作所花费的时间。
由于 n 个元素的全排列共有 n!种，如果程序输出所有排列，每个排列需要输出 n 个元素，所以总时间复杂度为：O(n×n!)

## 6. 实验总结

通过本次实验，我进一步理解了穷举法求解问题的基本思想。任务分配问题中，每个人只能选择一个任务，每个任务也只能被一个人选择，因此所有可能的分配方案可以看作任务编号的排列。本实验通过递归回溯的方法枚举所有合法分配方案，并在搜索过程中计算每种方案的总成本，最终得到最小总成本。
在程序实现过程中，我使用 used 数组记录任务是否已经被分配，从而保证每个任务只被使用一次。通过样例测试，程序输出的最小总成本为 13，与手工计算结果一致；通过 n=6 的新测试用例，程序输出结果也与分析结果一致，说明程序能够正确求解任务分配问题。
通过本次实验，我也认识到穷举法虽然思路清晰、实现简单，并且能够保证得到最优解，但它需要枚举所有可能的分配方案。当问题规模增大时，算法的运行时间会快速增加，效率较低。因此，穷举法更适合用于规模较小的问题。总体来说，本次实验加深了我对排列树、递归回溯和穷举法时间复杂度的理解。